Open vs. closed formulas

Definitions

- A variable x in a formula φ is free if x does not occur in the scope of any quantifier, otherwise it is bounded.
- An open formula is a formula that has some free variable.
- A closed formula, also called sentence, is a formula that has no free variables.

For closed formulas (but not for open formulas) we can define what it means to be true in an interpretation, written $\mathcal{I} \models \varphi$, without mentioning the assignment, since the assignment α does not play any role in verifying $\mathcal{I}, \alpha \models \varphi$.

Instead, open formulas are strongly related to queries — cf. relational databases.

FOL queries

Def.: A FOL query is an (open) FOL formula.

When φ is a FOL query with free variables $(x_1, \ldots, x_k)$, then we sometimes write it as $\varphi(x_1, \ldots, x_k)$, and say that φ has arity k.

Given an interpretation $\mathcal{I}$, we are interested in those assignments that map the variables $x_1, \ldots, x_k$ (and only those). We write an assignment α s.t. $\alpha(x_i) = a_i$, for $i = 1, \ldots, k$, as $\langle a_1, \ldots, a_k \rangle$.

Def.: Given an interpretation $\mathcal{I}$, the answer to a query $\varphi(x_1, \ldots, x_k)$ is

$$\varphi(x_1, \ldots, x_k)^{\mathcal{I}} = \{(a_1, \ldots, a_k) \mid \mathcal{I}, \langle a_1, \ldots, a_k \rangle \models \varphi(x_1, \ldots, x_k)\}$$

Note: We will also use the notation $\varphi^{\mathcal{I}}$, which keeps the free variables implicit, and $\varphi(\mathcal{I})$ making apparent that φ becomes a functions from interpretations to set of tuples.

FOL boolean queries

Def.: A FOL boolean query is a FOL query without free variables.

Hence, the answer to a boolean query $\varphi()$ is defined as follows:

$$\varphi()^{\mathcal{I}} = \{() \mid \mathcal{I}, \langle\rangle \models \varphi()\}$$

Such an answer is

- $()$, if $\mathcal{I} \models \varphi$
- $\emptyset$, if $\mathcal{I} \not\models \varphi$.

As an obvious convention we read $()$ as "true" and $\emptyset$ as "false".

FOL formulas: logical tasks

Definitions

- Validity: φ is valid iff for all $\mathcal{I}$ and α we have that $\mathcal{I}, \alpha \models \varphi$.
- Satisfiability: φ is satisfiable iff there exists an $\mathcal{I}$ and α such that $\mathcal{I}, \alpha \models \varphi$, and unsatisfiable otherwise.
- Logical implication: φ logically implies ψ, written $\varphi \models \psi$ iff for all $\mathcal{I}$ and α, if $\mathcal{I}, \alpha \models \varphi$ then $\mathcal{I}, \alpha \models \psi$.
- Logical equivalence: φ is logically equivalent to ψ, iff for all $\mathcal{I}$ and α, we have that $\mathcal{I}, \alpha \models \varphi$ iff $\mathcal{I}, \alpha \models \psi$ (i.e., $\varphi \models \psi$ and $\psi \models \varphi$).

FOL queries – Logical tasks

- Validity: if φ is valid, then $\varphi^{\mathcal{I}} = \Delta^{\mathcal{I}} \times \cdots \times \Delta^{\mathcal{I}}$ for all $\mathcal{I}$, i.e., the query always returns all the tuples of $\mathcal{I}$.
- Satisfiability: if φ is satisfiable, then $\varphi^{\mathcal{I}} \neq \emptyset$ for some $\mathcal{I}$, i.e., the query returns at least one tuple.
- Logical implication: if φ logically implies ψ, then $\varphi^{\mathcal{I}} \subseteq \psi^{\mathcal{I}}$ for all $\mathcal{I}$, written $\varphi \subseteq \psi$, i.e., the answer to φ is contained in that of ψ in every interpretation. This is called query containment.
- Logical equivalence: if φ is logically equivalent to ψ, then $\varphi^{\mathcal{I}} = \psi^{\mathcal{I}}$ for all $\mathcal{I}$, written $\varphi \equiv \psi$, i.e., the answer to the two queries is the same in every interpretation. This is called query equivalence and corresponds to query containment in both directions.

Note: These definitions can be extended to the case where we have axioms, i.e., constraints on the admissible interpretations.

Query evaluation

Let us consider:

- a finite alphabet, i.e., we have a finite number of predicates and functions, and
- a finite interpretation $\mathcal{I}$, i.e., an interpretation (over the finite alphabet) for which $\Delta^{\mathcal{I}}$ is finite.

Then we can consider query evaluation as an algorithmic problem, and study its computational properties.

Note: To study the computational complexity of the problem, we need to define a corresponding decision problem.

Query evaluation problem

Definitions

- Query answering problem: given a finite interpretation $\mathcal{I}$ and a FOL query $\varphi(x_1, \ldots, x_k)$, compute
$$\varphi^{\mathcal{I}} = \{(a_1, \ldots, a_k) \mid \mathcal{I}, \langle a_1, \ldots, a_k \rangle \models \varphi(x_1, \ldots, x_k)\}$$
- Recognition problem (for query answering): given a finite interpretation $\mathcal{I}$, a FOL query $\varphi(x_1, \ldots, x_k)$, and a tuple $(a_1, \ldots, a_k)$, with $a_i \in \Delta^{\mathcal{I}}$, check whether $(a_1, \ldots, a_k) \in \varphi^{\mathcal{I}}$, i.e., whether
$$\mathcal{I}, \langle a_1, \ldots, a_k \rangle \models \varphi(x_1, \ldots, x_k)$$

Note: The recognition problem for query answering is the decision problem corresponding to the query answering problem.

Query evaluation algorithm

We define now an algorithm that computes the function $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$ in such a way that $\texttt{Truth}(\mathcal{I}, \alpha, \varphi) = \texttt{true}$ iff $\mathcal{I}, \alpha \models \varphi$.

We make use of an auxiliary function $\texttt{TermEval}(\mathcal{I}, \alpha, t)$ that, given an interpretation $\mathcal{I}$ and an assignment α, evaluates a term t returning an object $o \in \Delta^{\mathcal{I}}$:

```
Δ^I TermEval(I,α,t) {
   if (t is x ∈ Vars)
      return α(x);
   if (t is f(t_1,...,t_k))
      return f^I(TermEval(I,α,t_1),...,TermEval(I,α,t_k));
}
```

Then, $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$ can be defined by structural recursion on φ.

Query evaluation algorithm (cont'd)

```
boolean Truth(I,α,φ) {
  if (φ is t_1 = t_2)
    return TermEval(I,α,t_1) == TermEval(I,α,t_2);
  if (φ is P(t_1,...,t_k))
    return P^I(TermEval(I,α,t_1),...,TermEval(I,α,t_k));
  if (φ is ¬ψ)
    return ¬Truth(I,α,ψ);
  if (φ is ψ ∘ ψ')
    return Truth(I,α,ψ) ∘ Truth(I,α,ψ');
  if (φ is ∃x.ψ) {
    boolean b = false;
    for all (a ∈ Δ^I)
       b = b ∨ Truth(I,α[x ↦ a],ψ);
    return b;
  }
  if (φ is ∀x.ψ) {
    boolean b = true;
    for all (a ∈ Δ^I)
       b = b ∧ Truth(I,α[x ↦ a],ψ);
    return b;
  }
}
```

Query evaluation – Results

Theorem (Termination of $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$)

The algorithm $\texttt{Truth}$ terminates.

Proof. Immediate. □

Theorem (Correctness)

The algorithm $\texttt{Truth}$ is sound and complete, i.e., $\mathcal{I}, \alpha \models \varphi$ if and only if $\texttt{Truth}(\mathcal{I}, \alpha, \varphi) = \texttt{true}$.

Proof. Easy, since the algorithm is very close to the semantic definition of $\mathcal{I}, \alpha \models \varphi$. □

Query evaluation – Time complexity I

Theorem (Time complexity of $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$)

The time complexity of $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$ is $O((|\mathcal{I}| + |\alpha| + |\varphi|)^{|\varphi|})$, i.e., polynomial in the size of $\mathcal{I}$ and exponential in the size of φ.

Proof.

- $f^{\mathcal{I}}$ (of arity k) can be represented as k-dimensional array, hence accessing the required element can be done in time linear in $|\mathcal{I}|$.
- $\texttt{TermEval}(\ldots)$ visits the term, so it generates a linear number of recursive calls, hence its time cost is $O(|\varphi| \cdot (|\mathcal{I}| + |\alpha|))$, i.e., polynomial time in $(|\mathcal{I}| + |\alpha| + |\varphi|)$.
- $P^{\mathcal{I}}$ (of arity k) can be represented as k-dimensional boolean array, hence accessing the required element can be done in time linear in $|\mathcal{I}|$.
- $\texttt{Truth}(\ldots)$ for the boolean cases simply visits the formula, so generates either one or two recursive calls.

Query evaluation – Time complexity II

- `Truth`$(\ldots)$ for the quantified cases $\exists x.\varphi$ and $\forall x.\psi$ involves looping for all elements in $\Delta^{\mathcal{I}}$ and testing the resulting assignments.
- The total number of such testings is $O(|\Delta^{\mathcal{I}}|^{\sharp\, Vars})$.

Considering that $O((|\varphi| \cdot (|\mathcal{I}| + |\alpha|)) \cdot |\Delta^{\mathcal{I}}|^{\sharp\, Vars}) \leq O(|\mathcal{I}| + |\alpha| + |\varphi|)^{(2+|\varphi|)})$, the claim holds. □

Query evaluation – Space complexity I

Theorem (Space complexity of $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$)

The space complexity of $\texttt{Truth}(\mathcal{I}, \alpha, \varphi)$ is $O(|\varphi| \cdot (|\varphi| \cdot \log |\mathcal{I}|))$, i.e., logarithmic in the size of $\mathcal{I}$ and polynomial in the size of φ.

Proof.

- $f^{\mathcal{I}}(\ldots)$ can be represented as k-dimensional array, hence accessing the required element requires $O(\log |\mathcal{I}|)$;
- $\texttt{TermEval}(\ldots)$ simply visits the term, so it generates a linear number of recursive calls. Each activation record has a size $O(\log |\mathcal{I}|)$ to evaluate the function call it represent, and we need $O(|\varphi|)$ activation records;
- $P^{\mathcal{I}}(\ldots)$ can be represented as k-dimensional boolean array, hence accessing the required element requires $O(\log |\mathcal{I}|)$;
- $\texttt{Truth}(\ldots)$ for the boolean cases simply visits the formula, so generates either one or two recursive calls, each requiring constant size;
- $\texttt{Truth}(\ldots)$ for the quantified cases $\exists x.\varphi$ and $\forall x.\psi$ involves looping for all elements in $\Delta^{\mathcal{I}}$ and testing the resulting assignments;

Query evaluation – Space complexity II

- The total number of activation records that need to be at the same time on the stack is $O(\sharp Vars)$.

Hence, we have $O(\sharp Vars \cdot (|\varphi| \cdot log(|\mathcal{I}|)) \leq O(|\varphi| \cdot (|\varphi| \cdot \log(|\mathcal{I}|))$ the claim holds. □

Note: the worst case form for the formula is

$$\forall x_1.\exists x_2.\cdots\forall x_{n-1}.\exists x_n.P(x_1, x_2, \ldots, x_{n-1}, x_n).$$

Query evaluation – Complexity measures [Var82]

Definition (Combined complexity)

The combined complexity is the complexity of $\{\langle \mathcal{I}, \alpha, \varphi \rangle \mid \mathcal{I}, \alpha \models \varphi\}$, i.e., interpretation, tuple, and query are all considered part of the input.

Definition (Data complexity)

The data complexity is the complexity of $\{\langle \mathcal{I}, \alpha \rangle \mid \mathcal{I}, \alpha \models \varphi\}$, i.e., the query φ is fixed (and hence not considered part of the input).

Definition (Query complexity)

The query complexity is the complexity of $\{\langle \alpha, \varphi \rangle \mid \mathcal{I}, \alpha \models \varphi\}$, i.e., the interpretation $\mathcal{I}$ is fixed (and hence not considered part of the input).